Progress Update: Rule Extraction MVP

Pipeline Modularization, Metadata Enrichment & Bug Fixes

April 5, 2026

---

1. Overview

This round of development focused on graduating the rule extraction pipeline from a proof-of-concept (ICDR Regulations 4–22, single monolithic script) to a modular, metadata-rich MVP architecture capable of scaling to the full ICDR (301 regulations, 12 chapters, 252 pages) and, eventually, to multiple regulatory frameworks.

Three areas were addressed:

1. **Modularization** : splitting the 1874-line `llm_extract_rules.py` into seven focused modules with clean interfaces.
2. **Metadata enrichment** : adding jurisdiction, chapter/part hierarchy, amendment tracking, cross-reference detection, and rule classification to every extracted rule.
3. **Bug fixes** : diagnosing and correcting seven extraction bugs identified through systematic validation of the v4 output (92 rules).

2. Output Validation: Bugs Identified in v4 Extraction

A detailed analysis of `rules_judged_v4.jsonl` (92 rules extracted from ICDR Regulations 4–22) revealed the following issues:

2.1 Critical Bugs

- **First-letter stripping in proviso/explanation rule IDs.** 31 of 92 rules contained truncated identifiers: `rovided` instead of `provided`, `xplanation` instead of `explanation`, `ategory` instead of `category`. Root cause: the tokenizer regex `(\d+[A-Z]?|[a-z]+)` matches either digits-with-optional-uppercase or lowercase-only. An uppercase initial letter (e.g., the "P" in "Provided") matches neither branch and is silently dropped.
- **Missing 7 key sub-clauses.** `ICDR_6_1_b` (operating profit $\geq$ 15 crore, the few-shot example itself), `ICDR_6_1_e`, `ICDR_5_1_d`, `ICDR_14_4`, `ICDR_10_1_a`, `ICDR_13_b`, and `ICDR_15_1_c` were absent from the output. Root cause: Pass 1 identification failure on dense pages where the LLM returns only a subset of visible clauses.

2.2 Medium Bugs

- **Regulation 8A encoded as `ICDR_8_8a`.** Regulation 8A is a standalone regulation, not a sub-clause of Regulation 8. The token detection loop treated `"8A"` as a non-digit token because `"8A".isdigit()` returns `False`, causing it to be appended as a suffix rather than recognized as the regulation number.
- **Full proviso text dumped into rule_id.** One rule ID reached 237 characters ; the entire proviso text was tokenized into underscore-separated words. Root cause: Pass 1 returned the full proviso text as `reg_number` instead of a structured clause identifier.

- **Duplicate field names across regulations.** `ofs_holding_period_years` appeared in 5 different rules (Reg 8, 16, 17, 19). Pass 2 extracts one clause at a time with no memory of previously assigned field names.
- **Continuation page context loss.** Sub-clauses on pages where the parent regulation header is not visible (e.g., Reg 6 sub-clauses continuing onto page 3 where only Reg 7's header appears) were either missed by Pass 1 or attributed to the wrong regulation.

2.3 Low–Medium Bugs

- **List Nat underrepresentation.** Only 2 of 73 typed fields used `List Nat`, despite many multi-year requirements in the ICDR. Rules containing "preceding three years" were typed as `Nat` or `Bool`.

3. Fixes Implemented

3.1 Proviso/Explanation Canonicalization

A `canonicalize_proviso_markers()` function was added to `regulation_identifier.py`. It normalizes natural-language proviso markers *before* the tokenizer runs:

- `Provided further` → `proviso2`
- `Provided that` → `proviso`
- `Provided` → `proviso`
- `Explanation` → `explanation`

Result: `ICDR_6_1_rovided` → `ICDR_6_1_proviso`; `ICDR_8_rovided_further` → `ICDR_8_proviso2`. Verified in first test run: all 31 affected rules now have clean IDs.

3.2 Rule ID Length Guard

A maximum length check was added to `normalize_rule_identifier()`: if the assembled rule_id exceeds 50 characters, only the first 5 tokens (including `ICDR` and the regulation number) are retained. The 237-character proviso-as-ID case is eliminated.

3.3 Regulation 8A Token Detection

The token detection loop in `normalize_rule_identifier()` was updated to recognize alphanumeric regulation numbers: `re.match(r"^\d+[A-Za-z]?$", tok)` replaces the previous `tok.isdigit()` check. This correctly identifies `"8A"` as the regulation token rather than a suffix.

3.4 Pass 1 Retry on Sparse Results

When Pass 1 returns fewer clauses than `pre_identify_regulations()` detected regulation numbers, a single retry is triggered with an explicit prompt listing the expected regulation numbers and the ones already found, asking the LLM to re-check for missing sub-clauses.

3.5 Continuation Page Context Carryover

The main extraction loop now carries forward `visible_regs` from the previous window into the current window's Pass 1 context. Window $[3, 4]$ receives not only Reg 7 (detected on page 3 by regex) but also Reg 6 (carried from window $[2, 3]$). This allows the LLM to correctly attribute Reg 6 sub-clauses that continue beyond their header page.

3.6 List Nat Deterministic Override

A post-extraction function `override_list_nat_from_text()` scans rule text for multi-year patterns ("preceding N years", "each of the N years") and overrides `Nat` type hints to `List Nat` with the appropriate length constraint. This is safe because the downstream schema reconciliation phase can always downgrade if evidence disagrees.

3.7 Field Name Deduplication

A `deduplicate_field_names()` post-processing pass detects field name collisions across rules and appends the regulation number as a disambiguator (e.g., `ofs_holding_period_years_reg8`, `ofs_holding_period_years_reg16`).

4. Modularization

The monolithic `llm_extract_rules.py` (1874 lines) was split into seven modules under `scripts/rule_extracti`

Module	Responsibility	Approx. Lines
`ollama_client.py`	Ollama HTTP client, JSON extraction, retry logic	250
`pdf_loader.py`	PDF reading, page windowing, page number stripping	60
`regulation_identifier.py`	Regex detection, Pass 1 LLM identification, normalization	350
`rule_extractor.py`	Prompts, few-shot examples, judge rubric, extraction	400
`rule_validator.py`	Schema validation, anchoring, span hint checks	300
`metadata_enricher.py`	Hierarchy lookup, jurisdiction, cross-references	300
`rule_store.py`	Persistence, versioning, deduplication	80

The orchestrator (`llm_extract_rules.py`) was reduced to approximately 250 lines: argument parsing, the window loop, and wiring the modules together.

5. Metadata Enrichment

Each extracted rule is now enriched with regulatory metadata via a new `metadata_enricher.py` module. An ICDR structure lookup table (`icdr_structure.json`) was built from the full 252-page SEBI ICDR 2018 PDF, mapping all 301 regulations to their parent chapter and part.

New fields added to every rule record:

Field Group	Fields
Identity & hierarchy	`regulation_framework`, `regulation_number`, `chapter`, `part`
Jurisdiction	`jurisdiction`, `regulator`, `country`
Classification	`rule_type`, `condition_type`, `compliance_actor`, `applicability_scope`
Amendment tracking	`original_effective_date`, `last_amended_date`, `amendment_history`, `is_current`
Cross-references	`references_regulations`, `references_schedules`, `references_external`
Proviso structure	`is_proviso`, `exception_to`
Pipeline metadata	`extraction_timestamp`, `extraction_model`, `pipeline_version`

All new fields are additive ; existing fields (`rule_id`, `text`, `maps_to`, `source`, etc.) are preserved unchanged. Downstream scripts (`schema_reconcile.py`, `llm_generate_lean.py`) continue to work without modification.

6. Debug Infrastructure

Three intermediate output files were added behind a `--debug-dir` flag:

- `pass1_inventory.jsonl` : one record per window showing regex-detected regulation numbers and LLM-identified clauses. Enables immediate diagnosis of Pass 1 misses.
- `pass2_pre_judge.jsonl` : every extracted rule before the judge loop modifies it. Enables diffing pre-judge vs. post-judge output.
- `coverage_summary.json` : final report listing all extracted rule IDs and flagging any regulations in the `--reg-filter` range with zero coverage.

A `--resume` flag was also added: when a run crashes mid-extraction, re-running with `--resume --append` reads the last processed window from `pass2_pre_judge.jsonl` and continues from the next window.

7. Validation Results

Initial test runs on the POC document (ICDR Regulations 4–22) with the refactored pipeline show:

- **Proviso fix confirmed:** Rule IDs now use `proviso`, `proviso2`, and `explanation` consistently. Zero instances of truncated words (`rovided`, `xplanation`).
- **Missing rules recovered:** `ICDR_6_1_b` (operating profit), `ICDR_5_1_d` (fugitive economic offender), `ICDR_5_1_explanation` now appear in the output.
- **Pass 1 identification improved:** Window [1,2] now identifies 12 clauses (vs. 1 in v4), including both provisos for Reg 6(1).
- **Continuation page fix in progress:** Reg 6 context carryover to window [3,4] is being validated.

8. Next Steps

1. Complete the POC validation run and diff against `rules_judged_v4.jsonl` to confirm all 7 bugs are resolved.
2. Run the full ICDR extraction (all 301 regulations, 252 pages) once the POC validates.
3. Integrate the enriched output with the downstream schema reconciliation and Lean code generation stages.
4. Evaluate whether the judge model (`deepseek-r1:14b`) needs to be replaced ; it consistently fails to produce valid JSON due to `<think>` tag wrapping.